

Major Progress Tracker

Our project helps students plan their classes by showing which classes they still need to complete the CS major and suggesting an appropriate schedule for the next semester. These recommendations are based on a dataset of offered classes and their prerequisites. While the main focus is completing the CS major on time, we also include non-CS classes in the suggested schedule.

Developed by Tom Burton, Henry Cannon, Phineus Choi, Deniz Tanaci.

[Github Repo Link](#)

Needfinding

- **What was your initial idea?**

Our initial idea was to create a better hyperschedule that would be more customizable. The idea was that we could “tune” it to students by their major, and then give them potential schedules based on which classes they wanted to take.

- **Who did you interview?**

We interviewed roommates and close friends

- **What were the takeaways you learned from your interviews? How did these takeaways influence what project you ended up making?**

The big takeaways we got were that we really needed to prioritize user-friendliness. Everyone we talked to had the major complaint that what made hyperschedule bad is its filters are difficult to use. This impacted our project because we limited the ways to interact with our program. Choosing options is as simple as inputting a number, as opposed to trying to navigate a complicated “About” or “Settings” section.

- **What problems did you find that are best not solved by software?**

Most of this is honestly software solvable to an extent, in that most people will want clearer filters, we can make clearer filters. The main thing software does not solve that comes up, is having our program interact with the actual portal website (ie, inputting a schedule on our program actually puts you in that schedule on the portal). There *could* be software to do that but obviously the schools are not going to allow it.

Our Dataset

- **A description of our dataset**

We chose to use the dataset format that we did because it allowed us to easily load the data at the start of the app in an easy, low cost way. We were able to format the data as two CSV files, a courses.csv and sections.csv using the CourseDataBuilder.java file. This file takes in the different files that are used for hyperschedule, parse them, and combine the data into the two CSV files. courses.csv contains the overall course details for each course offered in the following way: course id, course name, college, area code, cip code, course areas, and description. The sections.csv file contains the time and location details of the different offerings for each course in the following way: course id, section id, section number, semester id, session id, begin date, end date, credit hours, capacity, current enrollment, status, perm count, meeting days, meeting days raw, start time, end time, room, and instructors. We made this aspect of the project first, which meant that we are storing a bit more data for each section/course than used in the project. If the project was expanded though, it would allow us to build additional features without having to re-write the data scraping pieces of the project. This data then gets compiled into course and section info that we store in a hashmap

- **A short written justification on how you chose your data structure(s) for this project**

We chose to use a hashmap to store the course data because we did not need to maintain the order of courses and we wanted to have fast retrieval of a course based on their id. Additionally, we maintained the sections in an arraylist within each respective course. This meant that if we had a given course id, in $O(1)$ time we could locate the sections that related to the course and furthermore, could find a specific section in $O(1)$ time as there were never any courses that had > 5 sections.

Results

```
Please Enter A Number Option
Press:
1: to add classes taken
2: to remove a class from classes taken
3: to see the courses entered
4: to check your major progress
5: to get a possible schedule
6: reset data
7: exit app
1
Please Now Enter The Courses You Would Like To Add By ID, for example: CSCI054 P0
Enter Course To Add, or type 'STOP':
CSCI051 P0
CSCI051 P0
Course Added!
The Following Courses Were Added:
Introduction to Computer Science
Enter Course To Add, or type 'STOP':
```

```
Press:
1: to add classes taken
2: to remove a class from classes taken
3: to see the courses entered
4: to check your major progress
5: to get a possible schedule
6: reset data
7: exit app
2
Please Now Enter The Courses You Would Like To Remove By ID, for example: CSCI054 P0
Enter Course To Remove, or type 'STOP':
CSCI051 P0
Course Removed!
The Following Courses Were Removed:
Introduction to Computer Science
Enter Course To Remove, or type 'STOP':
```

```
Press:
1: to add classes taken
2: to remove a class from classes taken
3: to see the courses entered
4: to check your major progress
5: to get a possible schedule
6: reset data
7: exit app
3
The courses that you have taken are:
1: Introduction to Computer Science
2: Math Foundations of CS
Hit enter to continue
```

```
Press:
1: to add classes taken
2: to remove a class from classes taken
3: to see the courses entered
4: to check your major progress
5: to get a possible schedule
6: reset data
7: exit app
4
You still need to complete the following introductory/core courses for your major:
62
101
105
140
190
You need to complete 3 more elective courses.
Hit enter to continue
```

```
Press:
1: to add classes taken
2: to remove a class from classes taken
3: to see the courses entered
4: to check your major progress
5: to get a possible schedule
6: reset data
7: exit app
5
Please enter the courses you know you want to take (max 4).
Enter Course Code (e.g., CSCI051 P0), type 'DONE' to generate, or 'STOP' to cancel:
CSCI051 P0
Course added to your desired list!
Enter Course Code (e.g., CSCI051 P0), type 'DONE' to generate, or 'STOP' to cancel:
DONE
A possible schedule is:
Introduction to Computer Science
Animation: Art and Motion
Principles of Economic Analysis
Special Topics in Government
Hit ENTER to continue
```

Analysis

For each of your features:

- What pre-conditions are necessary for this feature to run? Are there any obvious edge cases?
- What is the time complexity? Why?
- What is the space complexity? Why?

Add and remove:

The pre-conditions are having the data files set up correctly, and then having said data correctly formatted into the hashtables.

There are a few edge cases that come up from user inputting courses. Such as: user inputs a course that doesn't exist, user inputs a course they have already inputted, or user tries to remove a course they never took.

Time complexity – Both of these run with while loops that are $O(N)$. This is because in a worst case scenario the user is trying to input every single course, so the while loop runs N times and each “loop” does $O(1)$ work. HOWEVER, there is a strong argument to make that since each user will only likely take 32 credits worth of courses (4 courses a semester for 8 semesters) we should call this $O(1)$, since it is very likely they will never be inputting more than ~ 32 courses regardless of how many total courses are offered.

Space Complexity – These are both also $O(N)$ for the same reason. They maintain ArrayLists within the Student Object that in theory, if every course was added to, would grow linearly as you linearly added courses. However, there is the same argument as with time complexity that it is very very unlikely a user will ever have in their arraylist more than 32 courses, regardless of how many total courses are offered.

Course data builder

The pre-conditions of CourseDataBuilder are that we need to have the following files in the data/ directory: alt-staff.json, calendar-session-section.java, calendar-session.json, course-area.json, course-areas-description.json, course-section-schedule.json, course-section.json, perm-count.json, section-instructor.json, staff.json. These files must be in the same format as the files found on the hyper schedule git hub. If they are not, then the files will not be parsed correctly and therefore, will not be usable to compile the data.

Time complexity: This feature runs in $O(N + M)$ time, where N is the number of characters in every file, and M is the number of objects that are needed to be created. This is because we need to parse every character in every file, and then for each object, we need to create a line for the CSV files.

Space Complexity: Overall, the space complexity is around $O(\text{Sum}(N))$ where N is the size of each file. This is because we need to load the data. So, we end up storing the complete amount of data.

Display courses taken

Display courses taken is a feature that uses the getter for the courses taken from Student. It then loops through them and writes them each to the terminal. Because of this, there is no precondition as if there are zero classes taken, it just won't print out any classes. There are no obvious edge cases because inputs come internally from Student.

Time complexity: The feature runs in $O(N)$ time where N is the amount of classes that the student has inputted that they have taken. This is because it loops through all the classes in one for loop and performs a constant operation on each. We can note that in practice, this runs at $O(1)$ time because at most a student takes around 32 courses in their time at Pomona which in practice runs at $O(1)$ time.

Space Complexity: The space complexity of this is $O(1)$ because we are not creating new elements, but simply accessing them.

Check major progress

We assume the inputted classes are in a valid format. That is, a course area code of four characters, course number of three digits, Special character if lab, a single space, followed by school code of two characters. An example is "CSCI062 PO". This is a reasonable assumption though, because any inputted class first goes through a check through the list of all classes, and thus is in this valid format.

An edge case was that if classes were taken at another Claremont College, they wouldn't count. So, we updated our checks so that any class including and after CS 62 should allow off-campus alternatives, whereas 51, 54 and 190 didn't. Another assumption we make is that any CS class that isn't one of 51,54,62,101,105,140,190 is counted as an elective. The two edge cases we have considered and fixed were CS 50, as well as any labs, because these should not be counted as electives.

Time Complexity: Our basic algorithm is store a hashmap of each required class (51,54,62,101,105,140,190) and whether it's taken (false by default). We then iterate through each class and if they correspond to any of these required classes, the hashmap gets updated appropriately. We later iterate through the hashmap to print. Counting electives is similar. That is, we iterate through a list of classes taken and update our hashmap of required classes, and then do another iteration with lookups to the hashmap. Thus, this function runs in $O(a+b)$ time where a is the number of required classes and b is the number of classes a student has taken. In practice, these will be bounded above by some small number like 40, but even in extreme scenarios, this won't go above 100. Thus, this method effectively takes $O(1)$ time.

Space complexity: As mentioned, there is only a hashmap of required classes storing whether they have been taken. Thus, it takes $O(a)$ space where a is the number of required classes, which effectively is $O(1)$ since the number of required classes cannot be too high.

Schedule Generator

The pre-conditions are that the Student object must be initialized with their completed course history, the allCourses HashMap must be fully populated with Course objects, and the Course objects must have their Section lists (with valid start times and meeting days) and prerequisite lists fully parsed and attached.

One edge case we checked were courses with no prereqs that did not make since for a general user to take such as varsity sports or grade specific courses (senior seminar, ID1, etc.) or zero credit courses such as labs. Another edge case is if somehow there were not enough valid courses (If a student has severe time conflicts, lacks prerequisites, or the catalog is too small) in which case we return a schedule with fewer than 4 courses.

Time Complexity: $O(n)$, where n is the total number of courses in the catalog. The algorithm copies the catalog values to a list and shuffles it, which takes $O(n)$ time. It then iterates through that list. Inside the loop, operations like checking prerequisites, validating time conflicts against the existing schedule, and

checking for labs take $O(1)$ time because the generated schedule size is strictly capped at 4, and course sections/prerequisites are trivially small constants. Thus, the loop scales linearly with the catalog size.

Space Complexity: $O(n)$. The generator creates a new ArrayList (availableCoursesList) from the allCourses.values() HashMap in order to use Collections.shuffle(). This list scales linearly with the number of courses in the catalog (n). The data structures storing the final schedule and scheduled sections are capped at size 4, meaning they only take $O(1)$ auxiliary space.

Affordance Analysis

- **Who benefits most from this software project? Who (if anyone) is left behind?**

We believe this project will most likely help students confused by the major requirements or overwhelmed with the course selection process. This may include international students who are unfamiliar with the American education system, as well FGLI students who may find it as helpful guidance. Also, while our project is specifically for Pomona students, a similar project at a bigger school with less access to high-quality advising may end up being more helpful.

One possible issue is that this project may make major requirements look stricter than they are. In many cases, for people in special circumstances, course offerings/major requirements may be updated. For example, a student may have taken the class before in a community college and there may be some ambiguity in whether that would be accepted as credit. So, having a very methodical, strict software may challenge students with non-traditional backgrounds.

- **How does your software reinforce or challenge norms or power structures?**

We fight socioeconomic imbalance as this allows anyone with access to our project to receive help for course selection. While sometimes advising is taken for granted at a well-resourced school like Pomona, a similar project at a bigger school with less access to high-quality advising may actually be very helpful. As mentioned, this tool might especially be helpful as international or FGLI navigate the overwhelming course selection process.

One possible issue that our project may reinforce is the strict major requirements. As mentioned above, for people in special circumstances, course offerings/major requirements may be updated. For example, a student may have taken the class before in a community college and there may be some ambiguity in whether that would be accepted as credit. So, having a very methodical, strict software may challenge students with non-traditional backgrounds. This tool should not replace human input and talking to faculty about policy updates.

- **What ways (if any) may your software be used maliciously—i.e., what actions does your software afford?**

Like we mentioned above, it may challenge students with non-traditional backgrounds. Also, having a mistake in recommendations may seriously confuse or harm students, especially because students tend to not question guidance from confident sounding software, e.g. trusting confident LLMs when they hallucinate.

- **Any other ethical reflections you would like to mention?**

As mentioned, this project should be used as a supportive tool, but shouldn't replace human decision-making. It does not replace human input, especially in the case of having a non-traditional background that may lead to changes in major requirements after discussion with the faculty.

Reflection

This project took our group around 25-30 hours, which roughly aligned with our expectations. The overall general structure and usage of our project largely stayed the same from our original idea, however we often made many changes between data structures. Often data structures such as DAGs or BSTs which we originally planned on implementing were replaced by simpler, and more easily implemented data structures such as ArrayLists, especially when they had to be manually constructed. We also ended up removing the feature of sign-in/passcodes and storing individual user data since we felt the challenge of ensuring security in passwords and personal info was outside the scope of this project. In general, it was a very positive experience to work on a bigger project in a group dynamic, and tasks and interaction was well handled.